

BUILDING EDUCATIONAL OPERATING SYSTEM SIMULATOR (EduOS)

Student Information

Name: CHRISTIAN VINTHENG

Registration Number: 25 311 351 023

Module Code: 351 CS 2104

Project Title: BUILDING EDUCATIONAL OPERATING SYSTEM SIMULATOR

1. Introduction of the project

EduOS is a mini educational operating system simulator which is developed with the purpose of practically demonstrate major operating system concepts through implementation in both C and Python programming languages such as CPU scheduling, process management, process control block, inter-process communication among others.

This project has been developed with the combination of low-level systems programming with high-level scheduling simulation and visualization which are C and Python programming languages. Through EduOS, many concepts such as process management, threading, synchronization, scheduling, IPC, and CPU performance analysis were implemented and tested practically.

The system was designed in two main parts:

1. C Core – Responsible for low-level operating system simulation.
2. Python Scheduler – Responsible for CPU scheduling simulation and visualization.

2. Project Objectives

- To simulate process management using PCB structures
- To implement CPU scheduling algorithms which are First come first serve, short job first, priority scheduling and Round robin.
- To demonstrate threading and synchronization which is how threads are executed and synchronized in the system.
- To show how race conditions works and how it is solved using fixed and mutex locking
- To implement thread pools
- To implement IPC using shared memory and pipes
- To visualize scheduling using Gantt charts and comparison graphs
- To integrate C and Python modules together using JSON file
- To gain practical experience with Git and GitHub

3. Project Structure

Operating-System-Assignment-1/

```
├── C_core/  
├── Python_scheduler/  
├── controller/  
├── Documents/  
├── README.md  
└── .gitignore
```

Where inside C_core there are the following files; ipc_module.c, process_manager.c, thread_manager.c, thread_manager_race.c, thread_manager_fixed.c, thread_pool.c, main_sim.c and a folder called include which has a file called eduos.h where every individual file has its functionality different from others.

Inside Python_scheduler they are the following file; scheduler_sim.py, requirements.txt, sample_processes.csv, gantt.py where every individual file has its functionality different from others.

Inside controller is where the main_controller.py is stored and it is used to execute the whole system together at once.

Documents is a folder that holds all documents that will either be used in the process or generated from the process.

4. Process Management

The system implemented process management using a PCB (Process Control Block). The PCB stored:

- PID
- Arrival Time
- Burst Time
- Priority
- Process State

The PCB was implemented using a C struct in eduos.h and exported into pcb_snapshot.json.

5. Thread Management

The project implemented multithreading using POSIX threads.

Race conditions were demonstrated in thread_manager_race.c where multiple threads updated the same shared variable simultaneously hence causes unpredictable outputs when the program executes since threads overwrite each other's outputs before reaching their completion time.

The issue was fixed in thread_manager_fixed.c using:
pthread_mutex_lock()

`pthread_mutex_unlock()`

A thread pool with worker threads and task queues was also implemented.

6. Inter-Process Communication (IPC)

IPC was implemented using:

- Shared Memory
- Anonymous Pipes

Shared memory enabled fast communication between processes while pipes enabled parent-child communication.

7. CPU Scheduling Algorithms

Implemented scheduling algorithms:

- FCFS – First Job is execute first before others & jobs waiting for CPU are put in a queue.
- SJF – This means shortest job in the list is executed first before others.
- Priority Scheduling – Process with higher priority are executed before other in order.
- Priority Scheduling with Ageing – This is used to prevent starvation of processes.
- Round Robin – Process are given constant time for each to execute.

Round Robin supported configurable quantum values.

8. Performance Metrics

The scheduler calculated:

- Completion Time
- Turnaround Time
- Waiting Time
- Response Time

Aggregate metrics included:

- Average WT – which is average time that is taken for a process to wait for a CPU.
- Average TAT – which is average time taken that a process take to finish execution process.
- Average RT
- CPU Utilization – Which simply indicate how much or CPU is used by a specific process.
- Throughput

9. Visualization

Matplotlib was used to generate:

- Gantt Charts
- Comparison Charts
- Timeline Animation

Generated images were saved in Documents/screenshots/.

10. Requirements and Installation

C Requirements:

- GCC
- MinGW

Python Requirements:

- matplotlib
- numpy
- tabulate

Install dependencies using:

```
pip install -r requirements.txt
```

11. Compiling and Running the Project

Compile C modules:

```
gcc main_sim.c process_manager.c thread_manager.c thread_manager_race.c  
thread_manager_fixed.c thread_pool.c ipc_module.c -o main_sim -linclude
```

Run Python scheduler:

```
python scheduler_sim.py --file sample_processes.csv
```

Generate Gantt charts:

```
python gantt.py --file sample_processes.csv
```

12. Design Decisions

Major design choices included:

- PCB implemented using C structs
- POSIX shared memory for IPC
- Fixed-size thread pool
- Multiple scheduling algorithms
- JSON bridge for C-Python integration

13. Learning Outcomes

The project improved understanding of:

- process management
- CPU scheduling
- synchronization
- race conditions
- IPC
- thread pools
- visualization
- Git and GitHub workflow

14. Conclusion

EduOS successfully demonstrated practical implementation of major operating system concepts using both C and Python.

The assignment improved understanding of operating system internals, scheduling, synchronization, concurrency, and software organization.